Emotion Detection & Learning Support Engine

An AI-powered learning companion that reads how a student feels about their study problem and responds with empathetic, personalised guidance.

Prepared By	Venu Gopal
Date	15 July 2026
Team / Submission ID	

Project Description

The Emotion Detection & Learning Support Engine is a Streamlit web application that helps self-paced learners get emotionally aware study support. A student picks a subject field and describes their problem in plain English; the app runs the text through two independent emotion classifiers — a BiLSTM model and a fine-tuned BERT model — compares their predictions side by side, and detects mixed/secondary emotions using keyword-boosted probability thresholds. The detected emotion, subject and problem text are then sent to Google's Gemini 2.5 Flash model, which drafts a short, encouraging, field-specific response with a concrete next step. If Gemini is unavailable, the app automatically falls back to a curated, rule-based response so the learner is never left without guidance. Every interaction is logged to CSV and summarised in a live analytics dashboard covering emotion distribution, the learner's emotional journey over a session, and field-wise emotion trends.

Scenarios

Scenario 1: A Computer Science student types "I'm confused about recursion" using the quick-example button. The BiLSTM and BERT models both predict *Confused* with high confidence; Gemini responds by acknowledging the confusion, suggesting the student trace through a small recursive example by hand, and recommending they revisit the base-case concept before tackling harder problems.

Scenario 2: A Mathematics student writes a longer, frustrated message about repeatedly failing a set of practice problems. The models detect *Frustrated* as the primary emotion with *Confused* as a secondary emotion above the mixed-emotion threshold. The dashboard displays both scores, and the AI response encourages a short break before revisiting the problem in smaller steps.

Scenario 3: A student exploring machine learning out of interest selects the "I'm curious about machine learning" quick example. The models predict *Curious*; Gemini responds with an encouraging suggestion to explore a related hands-on project, reinforcing the learner's engagement rather than treating it as a problem to fix.

Technical Architecture

The application uses a modular, layered architecture. The Streamlit presentation layer collects the field and problem text and renders predictions, AI responses and analytics. The application layer runs preprocessing (src/preprocessing.py) and coordinates two independent prediction services — a Keras BiLSTM model (src/bilstm_predict.py) and a HuggingFace/PyTorch BERT model (src/bert_predict.py) — whose outputs are refined by keyword boosting (src/keyword_enhancer.py) and mixed-emotion detection (src/mixed_emotion.py, src/utils.py). The result feeds src/gemini_helper.py, which builds a prompt and calls the Gemini API, with a rule-based fallback dictionary (EMOTION_RESPONSES) if the call fails. src/logger.py persists every analysed interaction to a CSV file that also powers the Plotly analytics dashboard rendered directly in app.py. (Note: the project does not use a relational database — interaction history is stored as CSV rows, so no ER diagram is required.)

Pre-requisites

- Python programming proficiency — Python Documentation (https://docs.python.org)
- Streamlit framework familiarity — Streamlit Documentation (https://docs.streamlit.io)
- Basic TensorFlow/Keras and PyTorch knowledge for working with the BiLSTM and BERT models

- HuggingFace Transformers familiarity — Transformers Documentation (https://huggingface.co/docs/transformers)
- Google Gemini API familiarity — Gemini API Documentation (https://ai.google.dev/)
- Version control with Git — Git Documentation (https://git-scm.com/doc)
- Development environment setup with a Python virtual environment (venv)

Project Workflow

Milestone 1: Data Preparation & Preprocessing

- Activity 1.1: Collect emotion-labelled text data (eng_dataset.csv, emotion-emotion_69k.csv) covering the five target emotions: Confused, Frustrated, Confident, Bored, and Curious.
- Activity 1.2: Build a text-cleaning pipeline in src/preprocessing.py — lowercasing, stripping URLs, mentions and non-alphabetic characters, tokenising with NLTK, and removing stop-words and single-character tokens.
- Activity 1.3: Define the EMOTION_RESPONSES dictionary as the fallback knowledge base, pairing each emotion with an emoji, an empathetic message and a recommended action.

Milestone 2: Emotion Classification Models

- Activity 2.1: Train a BiLSTM classifier on the preprocessed text, saving the model as models/bilstm/bilstm.keras along with its tokenizer.pkl and label_encoder.pkl, using a fixed max sequence length of 80 tokens.
- Activity 2.2: Fine-tune a BertForSequenceClassification model (models/bert_emotion_model_final) with its own label encoder for a second, independent emotion prediction.
- Activity 2.3: Implement predict_bilstm() and predict_bert() in src/, both returning a unified schema of {emotion, confidence, probabilities} so the two models can be compared directly in the UI.
- Activity 2.4: Add keyword-based probability boosting (src/keyword_enhancer.py) and secondary-emotion detection (src/mixed_emotion.py, threshold = 0.15) to make the primary prediction more robust and to surface mixed emotional states.

Milestone 3: Generative AI Integration

- Activity 3.1: Configure the Gemini API using google-generativeai, loading the API key securely from a .env file via python-dotenv (never hard-coded in source).
- Activity 3.2: Implement build_prompt() in src/gemini_helper.py to assemble a structured prompt from the field, problem, detected emotion and confidence, instructing Gemini to acknowledge the emotion, give a field-specific tip, suggest a next step, and encourage the student in simple English.
- Activity 3.3: Implement get_gemini_response() with exception handling that returns None on failure, letting app.py cleanly fall back to the rule-based EMOTION_RESPONSES entry for the detected emotion.

Milestone 4: Streamlit Application Development

- Activity 4.1: Build the main input panel in app.py — a subject-field selector, a problem text area with a 10-character minimum, and three quick-example buttons for common learner statements.
- Activity 4.2: Implement the model comparison view, showing BiLSTM and BERT results side by side with primary-emotion metrics, mixed-emotion callouts, and per-emotion probability bars.
- Activity 4.3: Wire up the AI response panel (Gemini output or fallback), the recommended-action callout, and a 'Regenerate Response' button that re-calls Gemini without re-running the classifiers.
- Activity 4.4: Add sidebar controls — toggling Gemini usage, toggling CSV saving, toggling analysis detail visibility, clearing session history, and a live interaction counter.

Milestone 5: Logging & Analytics Dashboard

- Activity 5.1: Implement save_to_csv() in src/logger.py to append every analysed interaction (field, problem, emotion, confidence, response, timestamp) to emotion_response_examples.csv.
- Activity 5.2: Build the analytics dashboard in app.py using Plotly Express — an emotion-distribution pie chart, an emotional-journey line chart across the session, and a field-vs-emotion grouped bar chart.
- Activity 5.3: Add a summary tab with total sessions, average confidence, most common emotion, and a raw history table for full transparency.

Milestone 6: Testing & Local Deployment

- Activity 6.1: Manually test each emotion path (confused, frustrated, confident, bored, curious) end to end, including the minimum-length validation and the Gemini-disabled fallback path.
- Activity 6.2: Record informal response-time observations for BiLSTM, BERT and the Gemini call (see Performance Testing) and confirm the app remains responsive on CPU-only hardware.
- Activity 6.3: Run the application locally with streamlit run app.py for demonstration; deployment to a public host (e.g. Streamlit Community Cloud) is identified as a future step.

Milestone 1 (Detail): Data Preparation & Preprocessing

This milestone focuses on getting clean, well-labelled text ready for both classifiers, and defining a safety net of fallback responses that the app can always fall back on.

Activity 1.2: Building the Preprocessing Pipeline

preprocess_text() in src/preprocessing.py performs the following steps before text ever reaches the BiLSTM model:

- Lower-cases the input text.
- Strips URLs (http/www patterns) and @mentions, and removes the # character from hashtags.
- Removes all non-alphabetic characters and collapses repeated whitespace.
- Tokenises with NLTK's word_tokenize, then removes English stop-words and single-character tokens.

Milestone 2 (Detail): Emotion Classification Models

Activity 2.3: Unified Prediction Schema

Both predict_bilstm() and predict_bert() return the same dictionary shape, which is what allows app.py to render them side by side without special-casing either model:

```
{
  "emotion": "Frustrated",
  "confidence": 78.42,
  "probabilities": {"Frustrated": 0.7842, "Confused": 0.1204, "..."}
}
```

Activity 2.4: Mixed Emotion Detection

detect_mixed_emotions() (src/mixed_emotion.py) sorts the probability dictionary, keeps the top score as the primary emotion, and reports any other emotion scoring at least 0.15 as a secondary emotion. app.py uses the simpler get_mixed_emotions() helper from src/utils.py for the same purpose when rendering the UI comparison panel.

Milestone 3 (Detail): Generative AI Integration

Activity 3.1: Securing the Gemini API Key

- Create a Google AI Studio / Gemini API key from https://aistudio.google.com.
- Create a .env file in the project root and add: GEMINI_API_KEY=your_copied_api_key_here
- src/gemini_helper.py loads it automatically at import time via load_dotenv() and genai.configure().
- Add .env to .gitignore so the key is never committed to the repository.

Activity 3.2: Prompt Construction

build_prompt() assembles a plain-English instruction telling Gemini to (1) acknowledge the student's emotion, (2) give one field-specific study tip, (3) suggest one next step, and (4) encourage the student — explicitly asking for simple English with no markdown formatting, so the response renders cleanly inside a Streamlit info box.

Milestone 4 (Detail): Streamlit Application Development

Home / Analysis Page

The single-page interface titled "Emotion Detection & Learning Support Platform" is organised into an input column (field selector, problem text area, quick examples) and a settings sidebar (Gemini toggle, CSV-save toggle, analysis-detail toggle, live dashboard summary, clear-history button).

Model Comparison Panel

After clicking 'Get AI Learning Help', two columns appear side by side: BiLSTM results on the left and BERT results on the right. Each shows either a single primary-emotion metric or, if a secondary emotion crosses the mixed-emotion threshold, an info callout listing all qualifying emotions with their percentages, followed by a full sorted list of emotion scores with progress bars.

AI Response / Fallback Panel

If Gemini responded successfully, its text is shown in an info box under "AI Learning Assistant", alongside the recommended action for the detected emotion and a "Regenerate Response" button. If Gemini did not respond, the app instead shows the curated emoji, message and recommended action from EMOTION_RESPONSES.

Milestone 5 (Detail): Logging & Analytics Dashboard

Once at least one interaction has been analysed in the session, a "Learning Analytics" section appears with three tabs: **Emotions** (a pie chart of emotion distribution and a line chart of confidence over the session), **Fields** (a grouped bar chart of emotion counts per subject field), and **Summary** (total sessions, average confidence, most common emotion, and the full session history table).

Milestone 6 (Detail): Testing & Local Deployment

Activity 6.1: Running the Application Locally

- Create and activate a virtual environment: python -m venv venv
- Install dependencies: pip install -r requirements.txt
- Add the Gemini API key to a .env file as described above.
- Start the app: streamlit run app.py
- Open the local URL Streamlit prints (default http://localhost:8501).

Activity 6.2: Manual Test Coverage

Each of the five target emotions was exercised at least once using both the quick-example buttons and custom free-text input, with and without the Gemini toggle enabled, to confirm both the AI-generated path and the rule-based fallback path behave correctly (see the Performance Testing document for detailed results).

Conclusion

The Emotion Detection & Learning Support Engine demonstrates how a lightweight sequence model (BiLSTM) and a larger contextual model (BERT) can be combined and cross-checked to detect a learner's emotional state from free text, and how that signal can drive a generative AI layer (Gemini 2.5 Flash) to deliver personalised, encouraging learning support instead of generic content. The rule-based fallback ensures the experience degrades gracefully rather than failing outright, and the built-in analytics dashboard gives the learner visibility into their own emotional trends over a study session. Development covered data preparation, dual model training/integration, generative AI prompting, a full Streamlit interface, and CSV-based logging and analytics. Looking ahead, the project has clear room to grow: persisting history beyond a single browser session, deploying to a public host such as Streamlit Community Cloud, adding automated tests for the prediction pipeline, and extending emotion coverage to more languages and subject areas (see Scalability & Future Plan for the detailed roadmap).